```
Output:
string(5) "65535"
int(65535)
string(41) "65535 is the largest 16-bit unsigned int."
int(65536)
```

When we perform a string operation (concatenation) using the integer `$int`, it is converted to an equivalent numeric string, and the expression evaluates to a string. Conversely, a mathematical operation (addition) using the string `$str` gives an integer value (and to achieve this, the integer equivalent of `$str` was used). PHP is quite happy to accept either type and convert as necessary. In this example, the integer *65535* was converted to the string `'65535'` during the expression `$int . ' is the largest 16-bit unsigned int.'`, and vice versa when we added *1* to `$str`.

It is important to note that when a variable's type is converted (to suit the context it is used in), the original variable remains unchanged. PHP will substitute a type-converted value in the expression, then perform the operation:

```
var_dump($int . ' is the largest 16-bit unsigned int.');
var_dump($int);
```

```
Output:
string(41) "65535 is the largest 16-bit unsigned int."
int(65535)
```

In the example above, we can see that `$int` is still an integer after being used in a string context.

PHP's type coercion is a high-level abstraction of equivalence. In reality the data are quite different when the machine performing the operations is taken into consideration. Let's see how these two values are really represented.

Different Types, Different Data

To get the values of the bytes used to store our 16-bit number, we can use PHP's built-in `dechex()` function. This function takes integer input and returns a string containing the hexadecimal representation of that number.